P3477R5
There are exactly 8 bits in a byte

Published Proposal, 2025-02-15

This version:
http://wg21.link/P3477r5

Author:
JF Bastien (Woven by Toyota)

Audience:
EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
github.com/jfbastien/papers/blob/master/source/P3477r5.bs

C++ Standard Base:
5d16ad051cad754b49057c456d64bf50c9180688

Table of Contents

Abstract: `8`ers gonna `8`.

1. Revision History

1.1. r0

[P3477R0] was the first published version of the paper, prompted by internet denizens nerd-sniping the author into writing the paper. There was much rejoicing.

1.2. r1

[P3477r1] was revised a month later, after the internet denizens read the paper and provided substantial feedback regarding exotic architectures, and pointing out some embarrassing typos. In that period, a few internet denizens showed once more that they don't read papers and only read the title, commenting on things that are already in the paper. The author would scold them here, but realizes the futility of even mentioning this shortcoming.

The C++ committee's Evolution Working Group also reviewed the paper, with the following outcome:

> Poll: D3477r1: There are exactly 8 bits in a byte: forward to CWG/LEWG for inclusion in C++26, removing the `intptr`/`uintptr` changes.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 9 | 17 | 3 | 4 | 0 |
>
> Result: consensus in favor

1.3. r2

Revision r1 was seen by the C++ committee's SG22 C/C++ Liaison Group, with the following outcome:

> We think WG14 might be interested too, perhaps with a change to hosted environments only. No concerns raised from SG22 perspective.

No other changes to the paper besides removing SG22 from the audience list.

1.4. r3

A lengthy LEWG email discussion took place. The salient points reflected in the updated paper are:

- Expansion of the discussion on `int16_t`, `int32_t`, `int64_t` and their `unsigned` variants, leading to a wording update to clarify the [**cstdint.syn**] changes, and adding changes to [**basic.fundamental**]. See § 6 ABI Break! 💥 and § 7 short is 16, int is 32, and more changes, and the updates in § 8 Wording.
- More motivation regarding complexity in § 3 Motivation.

An astute reader pointed out a missing editorial edit to an example hidden deep within [**dcl.init.list**]. It is now in § 8 Wording.

A section on other languages and their choice of bytes was added in § 5 Other languages.

LEWG has requested that EWG review the updated paper.

EWG did so in Hagenberg, and took the following poll:

> Poll: D3477R3: There are exactly 8 bits in a byte: Having heard the feedback from LEWG's Reflector review, as well as the response, EWG re-affirms its vote to forward D3477R3 to CWG/LEWG for inclusion in C++26.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 21 | 12 | 3 | 0 | 1 |
>
> Consensus in favor.

1.5. r4

LEWG, undeterred, saw [P3635R0] and [P3633R0] in Hagenberg. These papers were published 1.5h before r3 was initially scheduled to be presented.

The poll taken was:

> POLL: Forward "P3477R3: There are exactly 8 bits in a byte" to LWG for C++26.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 9 | 8 | 7 | 4 | 6 |
>
> No consensus.

r4 therefore removes all library components from this proposal, and only move forward with the language change. Library changes are expected to come at a later date.

1.6. r5

r4 was reviewed on Friday in EWG. The session was effectively a joint session, as many LEWG attendees joined the discussion.

The poll taken was:

> POLL: P3477r4 There are exactly 8 bits in a byte: forward to CWG for inclusion in C++26.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 14 | 18 | 14 | 14 | 8 |
>
> No consensus.

The paper's title is therefore erroneous: as far as WG21 is concerned, there are at least 8 bits per bytes. Maybe 9, 24, 16, 32, or maybe 2048. The author therefore expects that library and compiler implementations of C++ will finally support non-8-bit architectures—which they have failed to do for over a decade—now that WG21 (of which implementors are members) has clearly expressed the desire to do so.

§ 2. Rationale

C has the `CHAR_BIT` macro which contains the implementation-defined number of bits in a byte, without restrictions on the value of this number. C++ imports this macro as-is. Many other macros and character traits have values derived from `CHAR_BIT`. While this was historically relevant in computing's early days, modern hardware has overwhelmingly converged on the assumption that a byte is 8 bits. This document proposes that C++ formally mandates that a byte is 8 bits.

Mainstream compilers already support this reality:

- GCC sets a default value of `8` but no upstream target changes the default;
- LLVM sets `__CHAR_BIT__` to `8`; and
- MSVC defines `CHAR_BIT` to `8`.

We can find vestigial support, for example GCC dropped dsp16xx in 2004, and 1750a in 2002. Search the web for more evidence finds a few GCC out-of-tree ports which do not seem relevant to modern C++.

[POSIX] has mandated this reality since POSIX.1-2001 (or IEEE Std 1003.1-2001), saying:

> As a consequence of adding `int8_t`, the following are true:
>
> - A byte is exactly 8 bits.
> - `CHAR_BIT` has the value 8, `SCHAR_MAX` has the value 127, `SCHAR_MIN` has the value -128, and `UCHAR_MAX` has the value 255.
>
> Since the POSIX.1 standard explicitly requires 8-bit char with two's complement arithmetic, it is easier for application writers if the same two's complement guarantees are extended to all of the other standard integer types. Furthermore, in programming environments with a 32-bit long, some POSIX.1 interfaces, such as `mrand48()`, cannot be implemented if long does not use a two's complement representation.

To add onto the reality that POSIX chose in 2001, C++20 has only supported two's complement storage since [P0907r4], and C23 has followed suit.

The overwhelming support for 8-bit bytes in hardware and software platforms means that software written for non-8-bit bytes is incompatible with software written for 8-bit bytes, and vice versa. C and C++ code targeting non-8-bit bytes are incompatible dialects of C and C++.

Wikipedia quotes the following operating systems as being currently POSIX compliant (and therefore supporting 8-bit bytes):

- AIX
- HP-UX
- INTEGRITY
- macOS
- OpenServer
- UnixWare
- VxWorks
- z/OS

And many others as being formerly compliant, or mostly compliant.

Even StackOverflow, the pre-AI repository of our best knowledge (after Wikipedia), gushes with enthusiasm about non-8-bit byte architectures, and asks which exotic architecture the committee cares about.

This paper cannot succeed without mentioning the PDP-10 (though noting that PDP-11 has 8-bit bytes), and the fact that some DSPs have 16-bit, 24-bit, or 32-bit words treated as "bytes." These architectures made sense in their era, where word sizes varied and the notion of a byte wasn't standardized. Today, nearly every general-purpose and embedded system adheres to the 8-bit byte model. The question isn't whether there are still architectures where bytes aren't 8-bits (there are!) but whether these care about modern C++... and whether modern C++ cares about them.

The example which seems the most relevant of current architecture with non-8-bit-bytes is TI's TMS320C28x, whose compiler manual states:

> The TI compiler accepts C and C++ code conforming to the International Organization for Standardization (ISO) standards for these languages. The compiler supports the 1989, 1999, and 2011 versions of the C language and the 2003 version of the C++ language.

TI has expressed that it does not intend to support C++11 or later. They offer a migration guide from 8-bit bytes.

Another recent DSP which is sometimes brought up is CEVA-TeakLite. The latest generation, CEVA-TeakLite-4, only supports a C compiler.

Yet another potentially relevant architecture is SHARC, whose latest compiler manual explains which architectures support different architectural features in section "Processor Features", and whether the compiler option `-char-size[-8|-32]` is available. In any case, only C++03 and C++11 are supported, with significant features missing (but interesting anachronisms can be enabled, I recommend reading that section of the manual!).

A popular DSP which supports 8-bit bytes is Tensilica, whose compiler is based on clang.

Qualcomm provides three compilers that target their Kalimba series of DSPs. The `kcc`, `kalcc`, and `kalcc32` compilers all appear to be C compilers with no support for C++. These compilers support a 24 bit byte size according to Coverity's configuration, no public documentation seems available to confirm this.

An EDG representative has privately stated:

> I checked the configuration files that our customers share with us. One customer shared a configuration file that sets bytes to 32 as recently as 2022. This would configure a front end that supports C++17 and some C++20.

The author would happily retract this papers or change the proposal if hardware implementors expressed a desire to support modern C++ on their non-8-bit-per-byte hardware.

Does this proposal prevent new weird architectures from being created? Not really! These hypothetical new architectures would write their entire software stack from scratch with or without this paper, and would benefit from C23's `_BitInt` as standardized by [N2763] rather than have `char` and other types of implicit size.

§ 3. Motivation

Why bother? A few reasons:

- The complexity of supporting non-8-bit byte architectures sprinkles small but unnecessary burden in quite a few parts of language and library (see below);
- Compilers and toolchains are required to support edge cases that do not reflect modern usage;

- New programmers are easily confused and find C++'s exotic tastes obtuse;
- Some seasoned programmers joyfully spend time supporting non-existent platforms "for portability" if they are unwise, even writing FAQs about this which others then read and preach as gospel;
- Our language looks silly, solving problems that nobody has.

One reason not to bother: there still are processors with non-8-bit bytes. The question facing us is: are they relevant to modern C++? If we keep supporting the current approach where Bytes"R"Us, will developers who use these processors use the new versions of C++?

A cut-the-baby-in-half alternative is to mandate that `CHAR_BIT % 8 == 0`. Is that even making anything better? Only if the Committee decides to keep supporting Digital Signal Processors (DSPs) and other processors where `CHAR_BIT` is not `8` but is a multiple of `8`.

Another way to cut-the-baby-in-half is to mandate that `CHAR_BIT` be `8` on hosted implementations, and leave implementation freedom on freestanding implementations.

Regarding complexity, some committee members have been convinced by arguments such as:

- How can one write a truly portable serialization / deserialization library if the number of bits per bytes aren't known?
- Networking mandates octets, specifying anything networking related (as the committee is currently doing with [P3482R0] and [P3185R0]) without bytes being 8 bits is difficult.
- The Unicode working group has spent significant time discussing UTF-8 on non-8-bit-bytes architectures, without satisfying results.
- How do `fread` and `fwrite` work? For example, how does one handle a file which starts with `FF D8 FF E0` when bytes aren't 8 bits?
- Modern cryptographic algorithm and the libraries implementing them assume bytes are 8 bits, meaning that cryptography is difficult to support on other machines. The same applies to modern compression.

Overall, the members who brought these concerns seem to agree that architectures with non-8-bit-bytes are a language variant of C++, for which completely different code needs to be written. Combine this with hardware vendors expressing that they will not update the version of C++ that they support, and we conclude that the committee is maintaining a dead language variant.

§ 4. Impact on C

This proposal explores whether C++ is relevant to architectures where bytes are not 8 bits, and whether these architectures are relevant to C++. The C committee might reach a different conclusion with respect to this language. Ideally, both committees would be aligned. This papers therefore defers to WG14 and the SG22 liaison group to inform WG21.

§ 5. Other languages

Information on other languages:

- Java's virtual machine specification states "`byte`, whose values are 8-bit signed two's-complement integers".
- Rust primitive types only contain `u8`, and since Rust is implemented with LLVM it can only support 8-bit bytes.
- Python `bytes` are restricted to values in `[0, 256)`.
- C# `System.Byte` represents an 8-bit unsigned integer.
- Swift special use numeric types only contain `UInt8`, and since Swift is implemented with LLVM it can only support 8-bit bytes.
- JavaScript view using arrays only support 8-bit bytes.
- Go's basic types says "`byte` // alias for `uint8`".

C and C++ feel smug.

§ 6. ABI Break! 💥

This paper mandates that bytes be 8 bits, and mandates that the `[u]intN_t` typedefs from `cstdint` no longer be optional. Or rather, the `[u]intN_t` mandare was in r3 of the paper, but is gone from r4 of the paper. Nonetheless, the discussion of ABI breaks is kept for future references.

> But, this is an *ABI break*!!! 😱
>
> — the sentence that has ended countless C++ committee papers (see [P2137r0])

Readers who've made it thus far will *love* this puzzle, and should take a short break from reading the paper to consider "wut? how? an abi, in *my* bytes???". Go on, try to find the break!

<intermission>

Do you see it yet?

If you think "ah! Imagine an implementation where `short` was 32 bits and `[u]int16_t` was not defined! Then it would need to define `[u]int16_t` and would thus need to make `short` 16 bits.". Then you are indeed clever... but wrong! Because such an implementation could keep its `short`s and make `[u]int16_t` an *extended integer types* from [**basic.fundamental**]. You fell victim to one of the classic blunders! The most famous of which is, *'never get involved in a wording argument with Core,'* but only slightly less well-known is this: *'Never go in against a Standards Pedant when a paper's death is on the line!'*.

Any more educated guesses?

<intermission>

Alright clever reader, consider C23's `stdint.h`, section **Minimum-width integer types**, which states:

> The typedef name `int_least`*N*`_t` designates a signed integer type with a width of at least *N*, such that no signed integer type with lesser size has at least the specified width. Thus, `int_least32_t` denotes a signed integer type with a width of at least 32 bits.
>
> The typedef name `uint_least`*N*`_t` designates an unsigned integer type with a width of at least *N*, such that no unsigned integer type with lesser size has at least the specified width. Thus, `uint_least16_t` denotes an unsigned integer type with a width of at least 16 bits.
>
> If the typedef name int*N*_t is defined, `int_least`*N*`_t` designates the same type. If the typedef name uint*N*_t is defined, `uint_least`*N*`_t` designates the same type.

Imagine a platform where `short` is greater than 16 bits, and where `int_least16_t` is `short` (thus, greater than 16 bits, let's call them *jorts*), and which did not define `int16_t`. Such a platform would see an ABI break because it now needs to define an exact-width type `int16_t`, and therefore per C23 minimum-width integer type rules needs to change `int_least16_t` since `int16_t` now exists. Such a platform must either change the width of `short` (a **huge** ABI break), or define `int_least16_t` to be an extended integer type (thus, no longer `short`). The latter is arguably an ABI break, albeit a tiny one because who uses `int_least16_t`?

game over

...or is it?

Well, does such a platform, one with *jorts*, exist? The author cannot find evidence of such a platform, or equivalently odd platforms (say, with questionable choices for `int`). Is lack of evidence proof? No, but here is what information exists:

- GCC defines macros for exact width types and has extensive per-platform tests of its value.
- LLVM also has extensive tests on the underlying type representing `int16_t`, and the size of this type.

One can perform the same searches for 32- and 64-bit integer types. The hypothetical ABI break relies on a hypothetical platform with surprising integer types which we will henceforth call *int-bozons* (drop the *N* to obtain *int-bozos*). Were someone to observe an ABI break, which we hope will not require a superconducting super collider, then this paper should be revisited, and the severity of the discovery examined.

Clever readers of the evidence will have noticed an oddity... the AVR microcontroller sets `int16_t` to `int`! Why yes indeed. Isn't this paper an ABI break for AVR? No, because on AVR, `sizeof int` is `2`. A fun archeological dig will uncover an LLVM review which attempts to fix AVR to match the GCC definition, because LLVM used to define `int16_t` to `short` and GCC defined it to `int` (the actual fix came in a separate commit). The patch explains that the LLVM and GCC ABIs don't match, and thereby *breaks the LLVM ABI to match the GCC one*. Imagine that, a compiler *breaking ABI*. 2021 was a wild year.

The paper therefore concludes that questions of ABI breakage are put to rest for the purpose of this paper.

§ 7. `short` is 16, `int` is 32, and more changes

This is already a long paper, but some readers have asked and the paper must therefore answer:

> Could we make `short` 16 bits, `int` 32 bits, etc? After all, we are making `char` 8 bits!

Well, no. As we just re-discovered above, we cannot have nice things because doing the suggested change would be an **ABI break** for AVR. Do we want to do an ABI breaks? No, we'd never do such a thing.

Could we just make `short` 16 bits? To quote Sir Lancelot: "No, it's too perilous."

This paper therefore does not propose further changes. Motivated individuals, Sirs Galahads of sorts, are welcome to write a proposal, or file angry NB comments demanding change to `short`, `int`, and others. They might justify themselves with "Look, it's my duty as a knight to sample as much peril as I can," and the author would not begrudge them. However, for the purpose of this paper, we've got to find the Holy Grail (there are exactly 8 bits in a byte). Come on.

8. Wording

8.1. Language

Edit [**intro.memory**] as follows:

> The fundamental storage unit in the C++ memory model is the *byte* , which is a contiguous sequence of 8 bits . ~~A byte is at least large enough to contain the ordinary literal encoding of any element of the basic character set and the eight-bit code units of the Unicode UTF-8 encoding form and is composed of a contiguous sequence of bits, the number of which is implementation defined.~~ The memory available to C++ program consists of one or more sequences of contiguous bytes. Every byte has a unique address.
>
> ~~[*Note:* The number of bits in a byte is reported by the macro `CHAR_BIT` in the header `climits`. — *end note*]~~
>
> [*Note:* A byte is at least large enough to contain the ordinary literal encoding of any element of the basic character set and the eight-bit code units of the Unicode UTF-8 encoding form. — *end note*]

Edit [**basic.fundamental**] as follows:

Table ❑ — ~~Minimum width~~ Width [**basic.fundamental.width**]

Type	~~Minimum width~~ Width *N*
`signed char`	8
`short int`	at least 16
`int`	at least 16
`long int`	at least 32
`long long int`	at least 64

The width of each standard signed integer type shall ~~not be less than~~ match the values specified in [**basic.fundamental.width**]. The value representation of a signed or unsigned integer type comprises *N* bits, where N is the respective width. Each set of values for any padding bits [**basic.types.general**] in the object representation are alternative representations of the value specified by the value representation.

Except as specified above, the width of a signed or unsigned integer type is implementation-defined.

Edit the example at the end of [**dcl.init.list**] as follows:

```
int x = 999;                    // x is not a constant expression
const int y = 999;
const int z = 99;
char c1 = x;                    // OK, though it potentially narrows (in
char c2{x};                     // error: potentially narrows
char c3{y};                     // error: narrows ~~(assuming char is 8 bi~~
char c4{z};                     // OK, no narrowing needed
unsigned char uc1 = {5};        // OK, no narrowing needed
unsigned char uc2 = {-1};       // error: narrows
unsigned int ui1 = {-1};        // error: narrows
signed int si1 =
  { (unsigned int)-1 };         // error: narrows
int ii = {2.0};                 // error: narrows
float f1 { x };                 // error: potentially narrows
float f2 { 7 };                 // OK, 7 can be exactly represented as a
bool b = {"meow"};              // error: narrows
int f(int);
int a[] = { 2, f(2), f(2.0) };  // OK, the double-to-int conversion is nc
```

9. Wording for later papers

The following library wording changes, as of r4 of the paper, are not proposed by this paper. The changes below are kept in the paper for reference.

9.1. Library

Edit [**climits.syn**] as follows:

```
// all freestanding
#define CHAR_BIT ~~see below~~8
#define SCHAR_MIN ~~see below~~-128
#define SCHAR_MAX ~~see below~~127
#define UCHAR_MAX ~~see below~~255
#define CHAR_MIN see below
#define CHAR_MAX see below
#define MB_LEN_MAX see below
#define SHRT_MIN see below
#define SHRT_MAX see below
#define USHRT_MAX see below
#define INT_MIN see below
#define INT_MAX see below
#define UINT_MAX see below
#define LONG_MIN see below
#define LONG_MAX see below
#define ULONG_MAX see below
#define LLONG_MIN see below
#define LLONG_MAX see below
#define ULLONG_MAX see below
```

The header `climits` defines all macros the same as the C standard library header `limits.h`, except as noted above .

> Except for `CHAR_BIT` and `MB_LEN_MAX`, a macro referring to an integer type `T` defines a constant whose type is the promoted type of `T`.

Edit [**cstdint.syn**] as follows:

The header `cstdint` supplies integer types having specified widths, and macros that specify limits of integer types.

```
int8_t
int16_t
int32_t
int64_t
int_fast8_t
int_fast16_t
int_fast32_t
int_fast64_t
int_least8_t
int_least16_t
int_least32_t
int_least64_t
intmax_t
intptr_t
uint8_t
uint16_t
uint32_t
uint64_t
uint_fast8_t
uint_fast16_t
uint_fast32_t
uint_fast64_t
uint_least8_t
uint_least16_t
uint_least32_t
uint_least64_t
uintmax_t
uintptr_t
INTN_MIN
INTN_MAX
UINTN_MAX
INT_FASTN_MIN
INT_FASTN_MAX
UINT_FASTN_MAX
INT_LEASTN_MIN
INT_LEASTN_MAX
UINT_LEASTN_MAX
INTMAX_MIN
INTMAX_MAX
UINTMAX_MAX
```

```
INTPTR_MIN
INTPTR_MAX
UINTPTR_MAX
PTRDIFF_MIN
PTRDIFF_MAX
SIZE_MAX
SIG_ATOMIC_MIN
SIG_ATOMIC_MAX
WCHAR_MAX
WCHAR_MIN
WINT_MIN
WINT_MAX
INTN_C
UINTN_C
INTMAX_C
UINTMAX_C

// all freestanding
namespace std {
  using int8_t         = signed integer type; // optional
  using int16_t        = signed integer type; // optional
  using int32_t        = signed integer type; // optional
  using int64_t        = signed integer type; // optional
  using intN_t         = see below;           // optional

  using int_fast8_t    = signed integer type;
  using int_fast16_t   = signed integer type;
  using int_fast32_t   = signed integer type;
  using int_fast64_t   = signed integer type;
  using int_fastN_t    = see below;           // optional

  using int_least8_t   = signed integer type;
  using int_least16_t  = signed integer type;
  using int_least32_t  = signed integer type;
  using int_least64_t  = signed integer type;
  using int_leastN_t   = see below;           // optional

  using intmax_t       = signed integer type;
  using intptr_t       = signed integer type;   // optional

  using uint8_t        = unsigned integer type; // optional
  using uint16_t       = unsigned integer type; // optional
  using uint32_t       = unsigned integer type; // optional
```

```
  using uint64_t       = unsigned integer type; // optional
  using uintN_t        = see below;             // optional

  using uint_fast8_t   = unsigned integer type;
  using uint_fast16_t  = unsigned integer type;
  using uint_fast32_t  = unsigned integer type;
  using uint_fast64_t  = unsigned integer type;
  using uint_fastN_t   = see below;             // optional

  using uint_least8_t  = unsigned integer type;
  using uint_least16_t = unsigned integer type;
  using uint_least32_t = unsigned integer type;
  using uint_least64_t = unsigned integer type;
  using uint_leastN_t  = see below;             // optional

  using uintmax_t      = unsigned integer type;
  using uintptr_t      = unsigned integer type; // optional
}

#define INTN_MIN         see below
#define INTN_MAX         see below
#define UINTN_MAX        see below

#define INT_FASTN_MIN    see below
#define INT_FASTN_MAX    see below
#define UINT_FASTN_MAX   see below

#define INT_LEASTN_MIN   see below
#define INT_LEASTN_MAX   see below
#define UINT_LEASTN_MAX  see below

#define INTMAX_MIN       see below
#define INTMAX_MAX       see below
#define UINTMAX_MAX      see below

#define INTPTR_MIN       see below             // optional
#define INTPTR_MAX       see below             // optional
#define UINTPTR_MAX      see below             // optional

#define PTRDIFF_MIN      see below
#define PTRDIFF_MAX      see below
#define SIZE_MAX         see below
```

```
#define SIG_ATOMIC_MIN   see below
#define SIG_ATOMIC_MAX   see below

#define WCHAR_MIN        see below
#define WCHAR_MAX        see below

#define WINT_MIN         see below
#define WINT_MAX         see below

#define INTN_C(value)    see below
#define UINTN_C(value)   see below
#define INTMAX_C(value)  see below
#define UINTMAX_C(value) see below
```

The header defines all types and macros the same as the C standard library header `stdint.h` , except that none of the types nor macros for `8`, `16`, `32`, nor `64` are optional .

All types that use the placeholder *N* are optional when *N* is not `8`, `16`, `32`, or `64`. The exact-width types `intN_t` and `uintN_t` for *N* = `8`, `16`, `32`, and `64` ~~are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding typedef-names.~~ may be aliases for the standard integer types [**basic.fundamental**], or the extended integer types [**basic.fundamental**]. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef-name.

> The macros `INTN_C` and `UINTN_C` correspond to the typedef-names `int_leastN_t` and `uint_leastN_t`, respectively.

Within [**localization**], remove the 4 *mandates* clauses specifying:

> ~~`CHAR_BIT == 8` is `true`~~

Within [**cinttypes.syn**], do not change the list of `PRI` macros, and leave this paragraph as-is:

> Each of the `PRI` macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in [**cstdint.syn**]. Each of the `SCN` macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in [[**cstdint.syn**]] and has a suitable `fscanf` length modifier for the type.

§ References

§ Informative References

[N2763]

Aaron Ballman; et al. *Adding a Fundamental Type for N-bit integers*. 2021-06-21. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2763.pdf

[P0907r4]

JF Bastien. *Signed Integers are Two's Complement*. 6 October 2018. URL: https://wg21.link/p0907r4

[P2137r0]

Chandler Carruth, Timothy Costa, Hal Finkel, Dmitri Gribenko, D. S. Hollman, Chris Kennelly, Thomas Köppe, Damien Lebrun-Grandie, Bryce Adelstein Lelbach, Josh Levenberg, Nevin Liber, Chris Palmer, Tom Scogland, Richard Smith, David Stone, Christian Trott, Titus Winters. *Goals and priorities for C++*. 2 March 2020. URL: https://wg21.link/p2137r0

[P3185R0]

Thomas Rodgers. *A proposed direction for C++ Standard Networking based on IETF TAPS*. 14 December 2024. URL: https://wg21.link/p3185r0

[P3477R0]

JF Bastien. *There are exactly 8 bits in a byte*. 16 October 2024. URL: https://wg21.link/p3477r0

[P3477r1]

JF Bastien. *There are exactly 8 bits in a byte*. 21 November 2024. URL: https://wg21.link/p3477r1

[P3482R0]

Thomas W Rodgers, Dietmar Kuhl. *Proposed API for creating TAPS based networking connections*. 14 December 2024. URL: https://wg21.link/p3482r0

[P3633R0]

Murat Can Cagri. *A Byte is a Byte*. 2025-02-13. URL: https://isocpp.org/files/papers/P3633R0.pdf

[P3635R0]

Nevin Liber. *We shouldn't rush to require either CHAR_BIT==8 or (u)intNN_t*. 2025-02-13. URL: https://isocpp.org/files/papers/P3635R0.pdf

[POSIX]

The Open Group Base Specifications Issue 8. 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799/